BCS1540 Exam: Solutions and Grading Notes

May 2026

Q1

Grading Notes

a.

A counter example needs to have all the inputs, the $A[1]$, $A[2]$, etc. AND the t_1, t_2, etc.

Optimality is given by the NUMBER of applicants assigned (note: since we want to maximize the number of applicants, no applicants should be assigned to two time slots!).

For the given example as long as 2 time slots are used, the solution is optimal!

Optimality does not care about which applicants are assigned or which time slots are used. The example given has multiple optimal solutions, that is, solutions where two applicants are assigned time slots.

b.

The setup needs to introduce the greedy solution G, assumed non optimal, and an optimal solution O. (note: in the sample solution these are M and OPT respectively.)

They need to differ at some point.

3 cases need to be observed: t_j assigned to G and not O, to O and not G, and assigned to different Applicants in O and G. 99% of you only considered the third case.

The argument points: $0/2$ if flawed or contradictory reasoning. $1/2$ if incomplete reasoning on 2 cases or only 1 case considered. $2/2$ for at least 2 cases with correct reasoning.

c.

Do not introduce notation without defining it. It is better to just stay with the notation used in the question. For example, many people mixed up n and m, and many people thought that k was the number of starting times (instead of m).

Some people also seemed attempt to only implement one iteration of the greedy rule, but the objective here was to fully implement greedyB, not just one iteration. If we only wanted one iteration, we would have explicitly said so.

In the runtime analysis, many people treated $m = n$ or $m \leq n$. This is incorrect, you should just use them both, e.g., $O(nm)$.

A key part is that you need to keep track of which applicants are used (or just ensure that they are removed from the applicants still to be considered), many didn't do this.

To pick an applicant for a given start time, the time condition needs to be checked together with whether the applicant has been used.
Even though the time condition was explicitly given in the question text, many people still had this incorrect in their code :(

Sample Solution

a.
$A[1] = [0, 2]$, $A[2] = [1, 3]$, $t_1 = 0$, $t_2 = 1$.
GreedyA only assigns one applicant:

- assigns $A[1] \to t_2$, nothing to t_1.

But we can assign $A[1] \to t_1$ and $A[2] \to t_2$.
Thus greedyA is not optimal.

Generally any input with two applicants and two time slots such that $A[1].s \leq t_1 < A[2].s \leq t_2 < A[1].f < A[2].f$ breaks optimality of greedyA. If more applicants are involved with no other dedicated timeslot, they need to not be available at t_1 for the input to remain a valid counter example.

b.
Let M be the (applicant, start time) pairs matched in the greedy output.
Let OPT be an optimal set of pairings that is as similar as possible to M.
If they are the same, greedy is optimal.
Otherwise:

- Let j be the index of the first starting time where M and OPT disagree.
- Case 1: OPT uses t_j but M does not.
  - Since OPT uses t_j, when greedyB processed t_j, there was an applicant available. Thus M would have also used t_j. Contradiction.
- Case 2: M assigns $A[\ell]$ to t_j but OPT does not.
  - 2a: OPT does not use t_j.
    - Modify OPT by (re-)assigning $A[\ell]$ to t_j. This does not change the size of OPT, but does make it more similar to M. Contradiction.
    - Note: if $A[\ell]$ was not used in OPT, then OPT would not be optimal since $A[\ell] \to t_j$ can be added to OPT.
  - 2b: OPT uses t_j but with a different applicant, $A[\ell^*]$.
    - If $A[\ell]$ is not used in OPT, swap $A[\ell^*]$ for $A[\ell]$.
    - If $A[\ell]$ is used in OPT, say for time t_x, then by the greedy rule, $t_x \geq t_j$.
    - Change $A[\ell] \to t_x$ to $A[\ell] \to t_j$ (valid by greedy) and change $A[\ell^*] \to t_j$ to $A[\ell^*] \to t_x$ (valid because $A[\ell^*]$ has a later finish time than $A[\ell]$ and $t_j \leq t_x$).
    - Both lead to greater similarity with M. Contradiction.

c.
The following block of pseudocode implements greedyB.

Algorithm 1 Greedy Matching

```
boolean array USED, initially all false.  ▷ USED[i] := if an applicant has been used or not.
solution = {}.                  ▷ This will store the matched pairs of applicants and start times.
for j ← 1 to m do                                                          ▷ O(m) iterations
                                                                           ▷ process start time t_j
    for i ← 1 to n do                                                      ▷ O(n) iterations
                                ▷ consider applicant i, constant if-condition and add; O(1) time
        if not USED[i] and A[i].s ≤ t_j and t_j + 1 ≤ A[i].f then
                                                  ▷ this is the applicant with the earliest finish time
                                                                ▷ that can be scheduled and is not used
            solution.add((A[i], t_j))
            set USED[i] = true
            break
        end if
    end for
end for
```

Runtime: See also inline in the pseudocode.

Two nested loops: $O(m)$ iterations for the outer loop, $O(n)$ iterations for the inner, and $O(1)$ work in the inner loop.

Thus, $O(mn)$ total.

Q2

a.
$T(n) = 12T(n/3) + n \log n$
$n^{log_3 12} > n \log n$
Recurrence work wins, so overall $\Theta(n^{log_3 12})$ (accepted O or nothing instead of Θ)

b.
$T(n) = 9T(n/3) + n \log n$
$n^{\log_3 9} = n^2 > n \log n$
Recurrence work wins, so overall $\Theta(n^2)$ (accepted O or nothing instead of Θ, accepted $\log_3 9$)

Q3

Grading Notes

Many people misused the variables given in the question. For example, n is already defined as the number of towers (intervals), and ℓ is the length of the highway.
Try to avoid redefining such things! (this was also a problem in Q1)

This was almost the same question as on the test but just with weights added. Just like on the test, the highway is continuous, you need to cover all real valued $x \in [0, \ell]$, and the towers are not restricted to have integer valued a_i, b_i, c_i.

Many tried an approach similar to the knapsack DP, leading to non-polynomial runtime :(
Namely, if your table involves the length of the highway as a parameter, it is not polynomial size. In fact, since the a_i, b_i, c_i are not guaranteed to be integer valued, this will not work at all, but on this point we were lenient.
As noted in the question, this means you cannot get full points on this question.
For 3a this means at most 2/3, for 3b at most 6/8, for 3c you can still get full points.

The right way is to follow an approach similar to the telescope scheduling problem (it has time intervals and weights, much like we have here).

Also, this question is about showing that you know how to use Dynamic Programming.
For example, not only are greedy approaches that try to use cost per unit distance incorrect (they can easily be fooled), they are not valid answers to the question.

Sample Solution

a.
i. Table: OPT has one entry for each tower, that is, the entries are OPT[i], for $i \in \{1, \ldots, n\}$. Here i is the only parameter and it indicates that only the first i towers are considered.

ii. Value: OPT[i] := the minimum cost to cover up to b_i and we insist on using tower i.

iii. Final answer: $X = \min\{\text{OPT}[i] : i \in \{1, \ldots, n\} \text{ and } b_i = \ell\}$.

b.
In the base case we have zero towers, and cover none of the highway.
This gives us OPT[0] = 0.

Recursive Case: Either we use tower i or not.

- If $a_i > b_{i-1}$. This means tower i cannot extend any prior solution and we discard it.
  - OPT[i] = INF. (this ensures we don't use this tower)
- Otherwise, we look for the best prior solution that we can extend when insisting on using tower i.
  - $\text{OPT}[i] = \min(c_i + \min(\text{OPT}[j] : j < i \text{ and } b_j \geq a_i))$.

c.
The size of the table is $O(n)$ since we have one entry for each tower.

To compute an entry for each tower we look into all previous entries. This means it takes $O(n)$ time.
Thus to compute the table it takes $O(n^2)$ time.

To recover a solution we back track the table (as in the pseudocode below).

Algorithm 2 Backtracking

```
Starting from the final answer X
Find the last tower used
i ← n
while OPT[i] ≠ X do                          ▷ loop over each interval, O(n)
    i ← i − 1                                ▷ constant O(1)
end while
Report i                                     ▷ store the most recent tower added to the solution
for j ← i − 1 downto 1 do                    ▷ loop over each interval O(n)
    if OPT[j] = OPT[i] − c_i then            ▷ use interval j
        Report j                             ▷ constant, O(1)
        i ← j                                ▷ revise most recent tower
    end if
end for
```

Runtime for backtracking: $O(n)$. Namely, look back through the towers one time each with constant work per tower.

Total Runtime: $O(n^2)$ to build the table and $O(n)$ for backtracking. Thus $O(n^2)$ total.

Q4

Grading Notes

For a, many people put things like 'is there a minimum-sized vertex cover?', which is vacuous since of course there is always a minimum-sized one. Some answers didn't make clear that the thresholds are part of the problem input, which lead to a small deduction.
For c, it was important to explicitly specify a function from Vertex-cover instances to Independent-set instances.

Sample Solution

a.
Independent-set:
Input: (G, k), with G a graph and k an integer.
Output: YES if G contains an independent set of size $\geq k$, NO otherwise.
Vertex-cover:
Input: (G, k), G a graph and k an integer.
Output: YES if G contains vertex cover of size $\leq k$, NO otherwise.
b.
Certificate for a YES instance is a set of k vertices of G with no edges between them (both size $\geq k$ and no internal edges can be verified in polynomial time).
c.
We reduce Vertex-cover to Independent-set. Define a function f from Vertex-cover instances to Independent-set instances by $f((G, k)) = (G, |G| - k)$.
We now show that $f(l)$ is a YES-instance if and only if l is. Observe that $S \subseteq V(G)$ is a vertex cover if and only if $V(G) \setminus S$ is an independent set, so G contains a vertex cover of size at most k if and only if it contains an independent set of size at least $|G| - k$.

Q5

Grading Notes

a.
The example solution is just one possibility—there are many variants which are fine (for example, we could branch on drivers rather than locations). An important thing is that we have to backtrack over partial solutions involving all drivers—we can't fix the drivers' routes one by one.
b.
Using ILP for decision problems (with a trivial objective to just test feasibility) was discussed in the lectures, but since it wasn't emphasised issues with the objective weren't penalised.
c.
Again, since optimising a dummy objective for a decision problem wasn't emphasised, answers that phrased things in a more 'opimisation'y way weren't penalised.

Sample Solution

a.
Define partial solutions `a` as containing a route for each driver, starting but not necessarily ending at the depot.

```
dead(a):
  for d in a.drivers:
    if length(d.route) > L: return true
  return false

complete(a):
  for l in locations:
    if !(a.visited(l)): return false
  for d in drivers:
    if d.route.last != 0: return false
  return true

extend(a,l):
  b := emptyset
  for d in drivers:
    c := a
    c.drivers(d).route.apppend(l)
    b.add(c)
  return b

backtrack:
  stack X = {emptyroutes}
  while !(X.empty):
    a = X.pop
    l = min(a.unvisited)
    for b in extend(a,l):
      if dead(b): continue
      if complete(b): return b
      X.push(b)
```

b.

Maximise 1, subject to:
For each d: $\sum_{k,i,j} c_{i,j} x_{d,i,j,k} \leq L$ (each driver drives distance at most L)
For each $i \neq 0$: $\sum_{d,k,j} x_{d,i,j,k} \geq 1$ (each location is visited at least once)
For each d, each k and each $i \neq 0$: $\sum_j x_{d,k,j,i} - x_{d,k+1,i,j} = 0$ (a driver leaves a location if and only if they arrived at it, i.e. their path is continuous, except they can stop at the depot)
For each d: $\sum_j x_{d,1,0,j} = 1$, $\sum_{k>1,j} x_{d,k,0,j} = 0$ (each driver starts at the depot and doesn't leave it later)
For each d, i, j, k: $0 \leq x_{d,i,j,k} \leq 1, x_{d,i,j,k} \in \mathbb{Z}$.

c.

Fractional relaxation: drop the integrality constraints. Since we only dropped constraints, if the original ILP was feasible then so is the relaxed LP/the optimum of the relaxed LP is no worse than the optimum of the ILP.
For partial solution `a`, write `LP(a)` for the relaxed LP corresponding to `a` (i.e. with additional constraints setting the variables in accordance with `a`).
Replace the function `dead`:

```
dead(a):
  if opt(LP(a)) = -infty: return true
  else: return false
```